TUGAS KELOMPOK

II3230 – KEAMANAN INFORMASI

End-To-End Secure Message Delivery

Disusun oleh:

Favian Rafi Laftiyanto	18223036
Ahmad Evander Ruizhi Xavier	18223064

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

BAB I PENDAHULUAN **3**
BAB II TUJUAN TUGAS **4**
BAB III DESAIN SKENARIO **6**
3.1. Skenario di Sisi Alice (Pengirim) 6
3.2 Skenario di Sisi Bob (Penerima) 8
BAB IV TOPOLOGI/ARSITEKTUR SISTEM **10**
4.1. Topologi Jaringan 10
4.2. Arsitektur dan Alur Sistem 10
BAB V LANGKAH KERJA ALICE **12**
5.1. Persiapan Sistem (Prasyarat) 12
5.2. Eksekusi dan Input Data 12
5.3. Proses Pengamanan Data (Otomasi Sistem) 12
5.4. Pembungkusan dan Pengiriman Payload 13
BAB VI LANGKAH KERJA BOB **15**
6.1. Persiapan Sistem (Prasyarat) 15
6.2. Penerimaan Data Payload 15
6.3. Proses Dekripsi Data 15
6.4. Proses Verifikasi (Integritas dan Autentikasi) 16
6.5. Penyimpulan dan Pencatatan Log 17
BAB VII ALGORITMA YANG DIGUNAKAN **18**
7.1 Algoritma Symmetric Encryption – AES-256 18
7.2 Algoritma Asymmetric Encryption – RSA-2048 19
7.3 Algoritma Hash – SHA-256 21
7.4 Algoritma Digital Signature – RSA-PSS 22
BAB VIII FORMAT PAYLOAD YANG DIKIRIM **25**
BAB IX HASIL PENGUJIAN **28**
BAB X ANALISIS **31**
BAB XI KESIMPULAN **32**
REFERENSI **33**

DAFTAR GAMBAR

Gambar 4.1 Activity Diagram End-to-End Secure Message Delivery.....11
Gambar 9.1 Proses pengiriman pesan dari Alice.....29
Gambar 9.2 Proses penerimaan pesan dari Bob.....30
Gambar 9.3 Perbandingan hash yang dihitung Bob dan hash yang dikirim Alice.....30

BAB I

PENDAHULUAN

Dalam era digital saat ini, pertukaran informasi melalui internet memiliki risiko keamanan yang tinggi, mulai dari penyadapan (*eavesdropping*) hingga manipulasi data (*tampering*). Keamanan informasi tidak cukup hanya dengan menyembunyikan isi pesan, tetapi juga harus menjamin bahwa pesan tersebut tidak diubah oleh pihak ketiga dan benar-benar dikirim oleh pihak aslinya yang sah.

Tugas ini mengimplementasikan konsep *Hybrid Cryptography* atau sering disebut sebagai *Digital Envelope*. Pendekatan ini menggabungkan keunggulan kecepatan enkripsi simetris dengan manajemen kunci pada enkripsi asimetris. Untuk melengkapi aspek keamanan, mekanisme *Hashing* dan *Digital Signature* diterapkan untuk memenuhi empat pilar utama keamanan informasi, yaitu:

1. *Confidentiality* (Kerahasiaan): Menjamin bahwa hanya pemilik *private key* yang dapat membaca pesan melalui enkripsi.
2. *Integrity* (Integritas): Menjamin bahwa pesan tidak mengalami modifikasi selama transmisi melalui *hashing*.
3. *Authentication* (Autentikasi): Memastikan identitas pengirim yang sah melalui *Digital Signature*.
4. *Non-repudiation* (Nirsangkal): Mencegah pengirim menyangkal pengiriman pesan karena tanda tangan digital dibuat menggunakan kunci privat yang hanya dimiliki olehnya.

Laporan ini memaparkan simulasi teknis pengiriman pesan dari pengirim (Alice) ke penerima (Bob) menggunakan komunikasi antar pihak berbasis *Socket Programming* yang dikembangkan memanfaatkan bahasa pemrograman *Python*.

BAB II
TUJUAN TUGAS

Pelaksanaan tugas dan penyusunan laporan ini bertujuan agar dapat memahami dan mempraktikkan secara langsung konsep pengamanan komunikasi data secara *end-to-end secure message delivery*. Secara lebih rinci, tujuan dari tugas ini adalah:

1. Memahami alur pengamanan End-to-End secara sederhana: Menguasai konsep dan alur kerja pengamanan pesan dari sisi pengirim (Alice) hingga proses penerimaan pesan di sisi penerima (Bob) secara sistematis.
2. Membedakan fungsi enkripsi simetris dan asimetris: Memahami secara praktis perbedaan peran antara enkripsi simetris (AES-256) yang efisien untuk mengenkripsi isi pesan (data), dan enkripsi asimetris (RSA-2048) yang aman untuk mengenkripsi dan mendistribusikan kunci simetris (*key exchange*).
3. Memahami penggunaan *hash* untuk integritas data: Mengimplementasikan fungsi *Hash* (SHA-256) untuk memastikan bahwa pesan yang dikirim (plaintext/ciphertext) tidak mengalami perubahan atau modifikasi selama proses transmisi di jaringan.
4. Memahami penggunaan *digital signature* untuk autentikasi pengirim: Memahami dan mengimplementasikan penggunaan *Digital Signature* (menggunakan *Private Key* pengirim) untuk membuktikan keaslian identitas pengirim dan mencegah penyangkalan pengiriman pesan.
5. Mengimplementasikan skenario komunikasi aman antar dua Host/IP: Membangun skenario komunikasi antara dua *host* atau alamat IP menggunakan metode *Socket Programming*, sehingga simulasi pengiriman *payload* menyerupai kondisi jaringan sebenarnya.

6. Menjelaskan proses dekripsi, verifikasi *hash*, dan verifikasi tanda tangan digital: Mampu menjelaskan dan mendemonstrasikan proses di sisi penerima, yang meliputi dekripsi kunci simetris, dekripsi pesan, penghitungan ulang dan verifikasi *hash*, serta validasi tanda tangan digital.

BAB III

DESAIN SKENARIO

Skenario pada tugas ini menyimulasikan komunikasi satu arah yang aman (*end-to-end secure message delivery*) antara dua entitas, yaitu Alice sebagai pengirim pesan dan Bob sebagai penerima pesan. Skenario ini dirancang untuk memastikan kerahasiaan, integritas, dan autentikasi data menggunakan pendekatan kriptografi campuran simetris dan asimetris.

Komunikasi antara Alice dan Bob diasumsikan melewati suatu jaringan publik yang rentan terhadap berbagai ancaman keamanan, seperti penyadapan oleh pihak ketiga (*man in the middle attack*) atau manipulasi isi pesan (*tampering*) di tengah jalan. Oleh karena itu, pengamanan pesan dilakukan dalam skenario ini. Adapun berikut adalah keterangan lebih lanjut terkait skenario untuk masing-masing entitas.

3.1. Skenario di Sisi Alice (Pengirim)

Alice memiliki pesan yang bersifat rahasia yang harus dikirimkan kepada Bob secara aman melalui jaringan (*Socket Programming*). Tahapan yang dilakukan di sisi Alice adalah sebagai berikut:

1. Menentukan pesan *Plaintext*: Alice menyiapkan data atau pesan awal berupa teks biasa (*plaintext*). *Plaintext* dibuat mengandung informasi penting, adapun contoh pesan *plaintext* yang dikirim Alice adalah sebagai berikut:

"Kepada Bob, Dengan ini saya, Alice, mengonfirmasi bahwa transaksi dengan ID TXN-982374 telah berhasil diproses. Nomor rekening pengirim adalah 445678912 dan nomor rekening penerima adalah 998123456. Jumlah dana yang ditransfer sebesar Rp200.000.000, selesai pada 30 Maret 2026 pukul 14:23 WIB. Mohon pastikan bahwa detail transaksi ini tetap bersifat rahasia dan hanya dicatat untuk keperluan internal

proyek Alpha. Harap tidak membagikan informasi ini kepada pihak yang tidak berwenang. Hormat saya, Alice"

2. Pembuatan Kunci Simetris: Alice membuat sebuah kunci simetris secara acak (yang memanfaatkan algoritma AES-256 berukuran 32 *bytes*) beserta Initialization Vector (IV) berukuran 16 *bytes*.
3. Enkripsi Pesan (Enkripsi Simetris): Alice mengenkripsi *plaintext* menggunakan algoritma AES-256 menggunakan kunci simetris dan IV yang baru saja dibuat untuk menghasilkan *Ciphertext*.
4. Enkripsi Kunci Simetris (Enkripsi Asimetris): Karena Bob membutuhkan kunci AES untuk membuka *ciphertext*, Alice mengenkripsi kunci AES menggunakan *Public Key* milik Bob (dengan memanfaatkan algoritma RSA-2048) dengan skema OAEP/padding enkripsi asimetris optimal). Hal ini dilakukan untuk memastikan Bob dapat mengakses kunci AES tersebut.
5. Pembuatan Hash: Alice menghitung nilai *hash* menggunakan algoritma SHA-256. *Hash* dilakukan terhadap *Plaintext*. Hal ini dilakukan untuk membuktikan integritas pesan asli dari pengirim. Jika Bob berhasil mendekripsi pesan dan hasil *hash*-nya cocok, maka dapat dipastikan tidak terjadi perubahan pada isi ataupun makna dari pesan yang dikirim.
6. Pembuatan Digital Signature: Alice membuat tanda tangan digital atas nilai *hash* yang dihasilkan pada langkah sebelumnya menggunakan *Private Key* milik Alice (memanfaatkan algoritma RSA-PSS). Tanda tangan ini berfungsi sebagai bukti autentikasi dan *non-repudiation* dari pesan yang dikirim.
7. Pengiriman Data: Seluruh komponen hasil kriptografi tersebut dibungkus ke dalam format JSON dan dikirimkan sebagai satu *payload* utuh dari IP Alice menuju IP Bob melalui koneksi TCP/IP. Adapun format *payload* yang dikirim dapat ditinjau pada Bab VIII.

3.2 Skenario di Sisi Bob (Penerima)

Bob bertindak sebagai penerima yang menerima koneksi dan *payload* dari Alice. Setelah paket diterima, tahapan skenario yang dilakukan oleh Bob adalah:

1. Penerimaan Data: Bob menerima paket JSON dari alamat IP Alice.
2. Dekripsi Kunci Simetris: Bob mengekstrak kunci simetris dari *payload* dan mendekripsinya menggunakan *Private Key* milik Bob. Proses ini mengembalikan kunci AES asli yang dibuat oleh Alice.
3. Dekripsi Ciphertext: Ciphertext didekripsi menggunakan kunci AES hasil dekripsi dan parameter IV. Tahap ini dilakukan untuk mendapatkan kembali *Plaintext* awal yang dikirim Alice.
4. Verifikasi Integritas Pesan (*Hash*): Bob secara mandiri menghitung ulang *hash* SHA-256 dari *plaintext* yang baru saja didekripsi. Bob kemudian membandingkan *hash* hitungannya (lokal) dengan *hash* yang dikirim oleh Alice di dalam payload.
5. Verifikasi autentikasi (*Digital Signature*): Bob memverifikasi *signature* yang ada di dalam *payload* menggunakan *Public Key* milik Alice. Proses ini menggunakan nilai *hash* lokal yang telah divalidasi pada tahap sebelumnya.
6. Menarik kesimpulan: Berdasarkan seluruh proses yang telah dilakukan, Bob dapat mengetahui 3 hal:
    - Berhasil atau tidaknya dekripsi pesan: Jika kunci AES berhasil dibuka dan *ciphertext* kembali menjadi teks utuh, maka dekripsi pesan dianggap berhasil.
    - Apakah integritas pesan terjaga: Jika *hash* lokal sama persis dengan *hash* di *payload* yang diterima, maka integritas pesan dianggap terjaga.

- Apakah pesan benar berasal dari Alice: Jika fungsi verifikasi *digital signature* menggunakan *Public Key* Alice, maka pesan dianggap valid dan benar dikirim oleh Alice.

BAB IV

TOPOLOGI/ARSITEKTUR SISTEM

4.1. Topologi Jaringan

Sistem pengiriman pesan ini diimplementasikan menggunakan arsitektur *Client-Server* berbasis TCP/IP melalui metode *Socket Programming*. Dalam skenario ini, terdapat dua entitas yang saling berkomunikasi, yaitu:

- Alice (Pengirim): Bertindak sebagai inisiator koneksi yang akan menyambungkan diri ke alamat IP dan *port* milik Bob untuk mengirimkan paket data berbentuk *payload*.
- Bob (Penerima): Bertindak sebagai *listener* yang membuka *socket* dan menunggu koneksi masuk pada *port* tertentu.

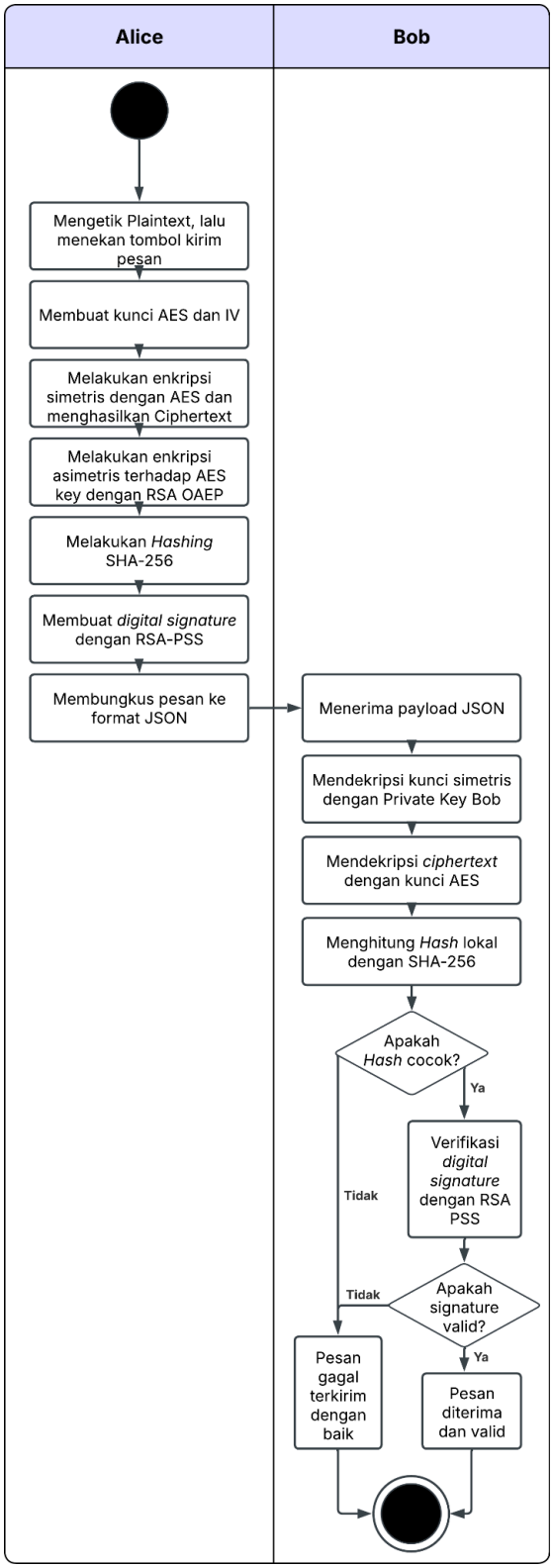
Untuk keperluan simulasi dan pengujian, komunikasi dilakukan dengan 2 variasi:

a. Menggunakan IP Virtual pada lingkungan lokal (*localhost*), dengan IP 127.0.0.1 dan 0.0.0.0.
b. Menggunakan IP dari suatu koneksi internet Wi-Fi yang digunakan oleh 2 buah perangkat berbeda.

Adapun protokol yang digunakan adalah TCP (*Transmission Control Protocol*) untuk memastikan pengiriman data yang andal (*reliable*) dan berurutan tanpa kehilangan paket dalam pengiriman pesan.

4.2. Arsitektur dan Alur Sistem

Berdasarkan topologi jaringan yang telah ditentukan, berikut adalah arsitektur dan alur sistem berbentuk *Activity Diagram* yang menggambarkan bagaimana pesan dikirim dari Alice hingga sampai ke Bob:



Gambar 4.1 *Activity Diagram End-to-End Secure Message Delivery*

BAB V

LANGKAH KERJA ALICE

5.1. Persiapan Sistem (Prasyarat)

Sebelum proses pengiriman pesan dapat dilakukan, sistem di sisi Alice harus memenuhi prasyarat berikut:

1. Ketersediaan Kunci Asimetris: Alice memastikan bahwa file *Private key* miliknya (*alice_private.pem*) dan file *Public Key* milik Bob (*bob_public.pem*) sudah tersedia di dalam direktori sistem lokal. Kunci ini dihasilkan dari tahap *Key Generation* sebelumnya.
2. Konektivitas Jaringan: Alice memastikan bahwa alamat IP dan *port* tujuan (IP Bob: 127.0.0.1 atau IP dinamis dari Wi-Fi, Port: 5005) dapat dijangkau dan siap menerima koneksi.

5.2. Eksekusi dan Input Data

Langkah kerja yang dilakukan langsung oleh pengguna (Alice) adalah sebagai berikut:

1. Alice menjalankan program pengirim (alice_gui.py).
2. Melalui antarmuka yang tersedia, Alice menentukan *plaintext* yang akan diamankan. Misalnya, Alice mengetikkan pesan *plaintext*.
3. Alice menekan tombol "Kirim Terenkripsi" untuk memicu seluruh fungsi kriptografi secara otomatis.

5.3. Proses Pengamanan Data (Otomasi Sistem)

Segera setelah tombol kirim ditekan, sistem Alice mengeksekusi tahapan pengamanan data (*end-to-end secure delivery*) secara berurutan sesuai dengan spesifikasi protokol:

1. Pembuatan Kunci Simetris: Sistem secara acak membangkitkan kunci sesi simetris (AES-256 *Key*) sepanjang 32 *bytes* dan *Initialization Vector* (IV) sepanjang 16 *bytes* menggunakan fungsi os.urandom().
2. Enkripsi Plaintext: Sistem melakukan *padding* pada pesan *plaintext* agar sesuai dengan ukuran blok AES (16 *bytes*). Selanjutnya, pesan dienkripsi menggunakan algoritma AES mode CBC dengan kunci dan IV yang baru dibuat, menghasilkan data biner Ciphertext.
3. Enkripsi Kunci Simetris: Sistem memuat Kunci Publik milik Bob (bob_public.pem). Kunci AES 32 *bytes* yang dibuat pada langkah pertama kemudian dienkripsi menggunakan algoritma RSA (dengan skema *padding* OAEP dan *hash* internal SHA-256), menghasilkan Encrypted Key.
4. Pembuatan Hash (Integritas): Sistem membaca *plaintext* awal dan memasukkannya ke dalam fungsi *hash* SHA-256. Proses ini menghasilkan nilai *digest* atau Hash berukuran 256-bit yang merepresentasikan sidik jari asli pesan.
5. Pembuatan Digital Signature (Autentikasi): Sistem memuat Kunci Privat milik Alice (alice_private.pem). Nilai *hash* yang dihasilkan pada langkah sebelumnya kemudian ditandatangani menggunakan algoritma RSA (dengan skema *padding* PSS). Proses ini menghasilkan data biner Digital Signature.

5.4. Pembungkusan dan Pengiriman Payload

Setelah seluruh komponen kriptografi berhasil dibuat, sistem Alice melakukan tahapan akhir:

1. Konversi Format: Mengingat *Ciphertext*, *IV*, *Encrypted Key*, *Hash*, dan *Signature* masih berupa data biner mentah (*raw bytes*), sistem mengonversi kelima

komponen tersebut ke dalam format string Base64 agar aman dikirimkan melalui protokol teks.

2. Pembentukan JSON: Kelima string *Base64* tersebut disusun menjadi satu objek kamus data (*dictionary*) dalam format JSON.
3. Transmisi Socket TCP: Sistem Alice membuka koneksi *Socket* TCP/IP menuju alamat IP dan *port* Bob. Setelah koneksi berhasil dibuat (*TCP Handshake*), paket JSON dikirimkan secara utuh.
4. Pencatatan Log: Aplikasi menampilkan log keberhasilan di layar Alice yang menandakan bahwa *payload* telah sukses dikirimkan dan sesi koneksi *socket* dari sisi pengirim ditutup.

BAB VI

LANGKAH KERJA BOB

6.1. Persiapan Sistem (Prasyarat)

Sebelum Bob dapat menerima pesan secara aman dari Alice, sistem di sisi Bob harus memenuhi prasyarat berikut:

1. Ketersediaan Kunci Asimetris: Bob memastikan bahwa file *Private Key* miliknya (*bob_private.pem*) untuk proses dekripsi, dan file Kunci Publik milik Alice (*alice_public.pem*) untuk proses verifikasi tanda tangan, sudah tersedia di direktori lokal.
2. Kesiapan Jaringan: Bob menjalankan program (*bob_gui.py*). Program ini menginisiasi *Socket* TCP dan melakukan *binding* pada alamat IP (0.0.0.0 atau alamat IP dinamis dari koneksi Wi-Fi) di *port* 5005. Sistem Bob akan berada dalam status menunggu koneksi masuk (*listening*).

6.2. Penerimaan Data Payload

Ketika Alice mengirimkan data, sistem Bob akan merespons dengan langkah berikut:

1. Sistem Bob menerima permintaan koneksi dari IP Alice dan melakukan *TCP Handshake* untuk membangun saluran komunikasi yang stabil.
2. Melalui saluran tersebut, Bob menerima aliran data (*byte stream*) berukuran hingga 8192 *bytes*.
3. Sistem mendeteksi bahwa data yang masuk adalah string berformat JSON. Sistem kemudian melakukan *parsing* (*JSON payloads*) untuk mengekstrak lima komponen utama: ciphertext, iv, encrypted_key, hash, dan signature.

6.3. Proses Dekripsi Data

Setelah paket *payload* berhasil diekstrak, sistem Bob memproses data tersebut secara berurutan di latar belakang:

1. Dekode Base64: Kelima komponen yang diterima masih dalam bentuk string Base64. Sistem mengonversinya kembali menjadi data biner mentah (*raw bytes*).
2. Dekripsi Kunci Simetris: Sistem memuat *Private Key* milik Bob. Kemudian, komponen *encrypted_key* didekripsi menggunakan algoritma RSA (dengan skema OAEP). Proses ini krusial karena mengembalikan Kunci AES (32 bytes) asli yang dibuat oleh Alice. Jika langkah ini gagal (misalnya karena kunci privat salah), seluruh proses selanjutnya akan dihentikan.
3. Dekripsi Pesan: Menggunakan Kunci AES yang baru saja didapatkan beserta parameter iv yang diterima dari *payload*, sistem Bob mendekripsi komponen ciphertext menggunakan algoritma AES-256 mode CBC. Setelah *padding* dihapus (*unpadding*), sistem berhasil memulihkan Plaintext awal (pesan asli dari Alice).

6.4. Proses Verifikasi (Integritas dan Autentikasi)

Setelah pesan berhasil dibaca, Bob belum dapat mempercayai pesan tersebut hingga melewati dua tahap verifikasi keamanan:

1. Verifikasi Integritas (Hashing): Sistem Bob mengambil *plaintext* yang baru saja didekripsi dan menghitung ulang nilai *hash*-nya secara mandiri menggunakan algoritma SHA-256. Sistem kemudian membandingkan *Local Hash* ini dengan nilai hash yang diterima di dalam *payload*. Jika kedua nilai tersebut identik, sistem mencatat di log bahwa integritas pesan terjaga.
2. Verifikasi Autentikasi (Digital Signature): Sistem memuat Kunci Publik milik Alice. Menggunakan Kunci Publik tersebut, sistem memverifikasi signature yang ada di dalam *payload* terhadap nilai *Local Hash* (menggunakan skema RSA-PSS). Jika fungsi verifikasi tidak mengembalikan pesan *error*, sistem

mencatat bahwa tanda tangan digital sah dan pesan benar-benar berasal dari Alice.

6.5. Penyimpulan dan Pencatatan Log

Sebagai langkah akhir, antarmuka pengguna (GUI) di sisi Bob akan menampilkan:

1. Pesan *plaintext* yang berhasil diungkap.
2. Status verifikasi *Hash* (misalnya: Integritas Pesan (Hash): VALID).
3. Status verifikasi *Signature* (misalnya: Digital Signature: VALID).

BAB VII

ALGORITMA YANG DIGUNAKAN

Seperti yang sudah dijelaskan pada bab-bab sebelumnya, algoritma yang digunakan untuk kriptografi simetris, asimetris, fungsi hash, dan digital signature adalah sebagai berikut:

7.1 Algoritma Symmetric Encryption – AES-256

Algoritma Advanced Encryption Standard (AES) merupakan algoritma kriptografi simetris berbasis *block cipher* dengan ukuran blok tetap 128-bit. AES-256 dipilih karena beberapa alasan berikut:

- Memiliki tingkat keamanan yang tinggi
- merupakan standar industri yang umum digunakan
- Efisien untuk enkripsi data dalam jumlah besar
- Lebih cepat dibandingkan algoritma asimetris

Pada sistem ini digunakan AES dengan panjang kunci 256 bit dalam mode CBC (Cipher Block Chaining). Pada mode CBC, setiap blok plaintext akan di-XOR dengan ciphertext blok sebelumnya sebelum dienkripsi sehingga menciptakan ketergantungan antar blok dan dapat meningkatkan keamanan. Enkripsi blok pertama memerlukan blok semu yang disebut *Initialization Vector* (IV) sebagai pengganti blok sebelumnya. Pada implementasi yang dilakukan, IV dibangkitkan secara acak oleh program. Sebelum dienkripsi, plaintext dipadding agar memiliki panjang kelipatan 16 byte sesuai ukuran blok AES. Berikut adalah implementasi enkripsi plaintext dengan AES-256 yang dilakukan oleh Alice berdasarkan penjelasan sebelumnya:

```
# 1. AES Key Generation
aes_key = os.urandom(32)
iv = os.urandom(16)
self.log(f"[STEP 1] Membuat AES-256 Key: {aes_key.hex()[:20]}...")

# 2. Symmetric Encryption (AES-CBC)
pad_len = 16 - (len(plaintext) % 16)
padded_plaintext = plaintext + bytes([pad_len] * pad_len)
cipher = Cipher(algorithms.AES(aes_key), modes.CBC(iv))
encryptor = cipher.encryptor()
ciphertext = encryptor.update(padded_plaintext) +
encryptor.finalize()
self.log(f"[STEP 2] Ciphertext:
{base64.b64encode(ciphertext).decode()[:30]}...")
```

Kunci AES yang dihasilkan akan digunakan oleh Bob (setelah kunci AES terdekripsi) untuk mendekripsi ciphertext dengan implementasi sebagai berikut:

```
# 2. Decrypt Ciphertext
ciphertext = base64.b64decode(payload['ciphertext'])
iv = base64.b64decode(payload['iv'])
cipher = Cipher(algorithms.AES(aes_key), modes.CBC(iv))
decryptor = cipher.decryptor()
padded_pt = decryptor.update(ciphertext) + decryptor.finalize()

pad_len = padded_pt[-1]
plaintext = padded_pt[:-pad_len]
self.log(f"[STEP 2] Plaintext Didekripsi: {plaintext.decode()}")
```

7.2 Algoritma Asymmetric Encryption – RSA-2048

Algoritma RSA merupakan algoritma kriptografi asimetris yang menggunakan pasangan kunci, yaitu *public key* dan *private key*. RSA bekerja berdasarkan faktorisasi bilangan bulat yang besar. RSA dipilih dengan beberapa alasan atau pertimbangan berikut:

- Sulit untuk memfaktorkan bilangan bulat yang besar menjadi faktor-faktor prima sehingga sangat aman untuk digunakan (kriptanalisis sangat sulit atau bahkan hampir mustahil untuk dilakukan).
- Memungkinkan distribusi kunci secara aman tanpa berbagi *secret* sebelumnya (Alice melakukan enkripsi plaintext dengan *public key* Bob yang tidak rahasia).
- Cocok untuk enkripsi kunci simetris karena ukurannya tidak terlalu panjang.

Pembuatan *public key* dan *private key* milik Alice dan Bob dilakukan dengan fungsi berikut:

```
def generate_keys(name):
    private_key = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
    # Simpan Private Key
    with open(f"{name}_private.pem", "wb") as f:
        f.write(private_key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.PKCS8,
            encryption_algorithm=serialization.NoEncryption()
        ))
    # Simpan Public Key
    with open(f"{name}_public.pem", "wb") as f:
        f.write(private_key.public_key().public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ))

generate_keys("alice")
generate_keys("bob")
print("Kunci RSA untuk Alice dan Bob berhasil dibuat.")
```

Dari sisi Alice, *symmetric key* dienkripsi dengan *public key* Bob melalui skema *padding* OAEP karena membuat enkripsi lebih aman, acak, dan tahan terhadap

berbagai jenis serangan. Implementasi tersebut dapat dilihat pada potongan kode berikut:

```
# 3. Encrypt AES Key with Bob's Public Key
with open("bob_public.pem", "rb") as f:
    bob_pub = serialization.load_pem_public_key(f.read())
enc_key = bob_pub.encrypt(
    aes_key,
    padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
)
self.log("[STEP 3] AES Key telah dienkripsi dengan RSA Public Key
Bob.")
```

Kemudian dari sisi Bob, *symmetric key* didapat dengan melakukan dekripsi *encrypted symmetric key* menggunakan *private key* Bob yang implementasinya dapat dilihat pada potongan kode berikut:

```
# 1. Decrypt AES Key
with open("bob_private.pem", "rb") as f:
    bob_priv = serialization.load_pem_private_key(f.read(),
password=None)

enc_key = base64.b64decode(payload['encrypted_key'])
aes_key = bob_priv.decrypt(
    enc_key,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
)
self.log("[STEP 1] AES Key berhasil didekripsi menggunakan Private
Key Bob.")
```

7.3 Algoritma Hash – SHA-256

Fungsi hash SHA-256 merupakan bagian dari SHA-2 yang menghasilkan nilai hash sepanjang 256-bit. Fungsi ini bersifat satu arah sehingga tidak memungkinkan untuk mengembalikan nilai asli dari hash. SHA-256 dipilih karena:

- Tahan terhadap *collision* sehingga memiliki keamanan yang tinggi.
- Tidak bisa dibalik (*one-way function*) sehingga tidak memungkinkan untuk mengembalikan nilai asli dari hash.
- Cepat dan efisien sehingga cocok untuk pengiriman dan penerimaan pesan.

Alice melakukan hashing pada plaintext untuk verifikasi integritas pesan dengan implementasi sebagai berikut:

```
# 4. Hashing
digest = hashes.Hash(hashes.SHA256())
digest.update(plaintext)
msg_hash = digest.finalize()
self.log(f"[STEP 4] Hash SHA-256: {msg_hash.hex()}")
```

Bob akan menerima hasil hash plaintext oleh Alice dan membandingkannya dengan hashing yang ia lakukan pada plaintext yang sudah didekripsi. Berikut implementasinya:

```
received_hash = base64.b64decode(payload['hash'])
digest = hashes.Hash(hashes.SHA256())
digest.update(plaintext)
local_hash = digest.finalize()

hash_status = "VALID :white_check_mark:" if local_hash ==
received_hash else "INVALID :x:"
self.log(f"[STEP 3] Integritas Pesan (Hash): {hash_status}")
```

7.4 Algoritma Digital Signature – RSA-PSS

RSA-PSS adalah sebuah skema tanda tangan digital berbasis RSA dengan rancangan lebih aman dibandingkan skema tanda tangan RSA klasik. Alasan pemilihan RSA-PSS untuk tanda tangan digital adalah sebagai berikut:

- Tahan terhadap serangan adaptif seperti *chosen-message attack* sehingga lebih aman untuk digunakan.
- tanda tangan untuk pesan yang sama akan berbeda setiap kali dibuat secara acak dan berbasis probabilitas (*probabilistic padding*).
- Terstandarisasi dalam praktik modern.

Alice membuat tanda tangan digital dengan menggunakan *private key* miliknya sebelum dikirimkan ke Bob dengan implementasi berikut:

```
# 5. Digital Signature
with open("alice_private.pem", "rb") as f:
    alice_priv = serialization.load_pem_private_key(f.read(),
password=None)
sig = alice_priv.sign(
    sg_hash,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)
self.log("[STEP 5] Digital Signature berhasil dibuat dengan Private
Key Alice.")
```

Kemudian Bob melakukan verifikasi tanda tangan digital tersebut menggunakan *public key* milik Alice dengan implementasi berikut:

```
 # 4. Signature Verification
with open("alice_public.pem", "rb") as f:
    alice_pub = serialization.load_pem_public_key(f.read())

sig = base64.b64decode(payload['signature'])
try:
    alice_pub.verify(
    sig, local_hash,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)
```

```
    self.log("[STEP 4] Digital Signature: VALID :white_check_mark:
(Pesan otentik dari Alice)")
except:
    self.log("[STEP 4] Digital Signature: INVALID :x:")
```

BAB VIII

FORMAT PAYLOAD YANG DIKIRIM

Pada sistem ini, Alice mengirimkan data ke Bob dalam bentuk **struktur JSON** yang berisi komponen-komponen hasil proses algoritma kriptografi sebelumnya. Payload ini dirancang agar pesan yang dikirim dapat digunakan Bob untuk dienkripsi/diverifikasi dengan memenuhi aspek *confidentiality, integrity,* dan *authentication*.

Berikut struktur payload yang digunakan:


```
{'ciphertext':
'p2PElJH89WcsY2rxJDUvrbOJUd07kzNBovYeoZpor1JbB0CBYJ9ojeqYajexetEQA/
epi4K5arhXTg5FyX1Awm+UyyuHle8DO8cwaCfjb7vyiA9SATIghZ+f48mW1VGcxu/PF
igP4JrjWGahAXJKhcYZRvsJCeyXefnUa8mLGOjtWp26g8COLWDjGeiJEhdAvtLAmIKL
dETsyF0mu0i+ACgMqKT7Ylivkz6kPWKldN12jLTGmSIR/nAhv12gbMz6lp4he8XRf4Z
aQYh6rCP/UHIO+yOg1RSPMbmjch6AnJfo7uXtzOaHaaiVm8Ab7hT06EqWrIT8k+fbrv
wXYfZ3aIjA6prdpAv8UsF02g9ro1hSlu03Vr0STcJ26+/6QflD6rqch1WepNXo8y8rU
rQRDS6+JeHuso/Y+KNU5QLXV+jR36R32z/YQZpGsWo3Rj5nrxNAigb0xU2HMrGAAzDc
HLemJNAnXDNTLxOS8M6iB76fMAuCWoMNMC1rMHZj821vLwFMtxoQnnGeYkns2uV1f4m
0T6ICI2OuBvObtz6l7p+yz6aGZeodoXyFXyUPgZnyNl48l9hD6xIqHflxiQxr7v2Nm6
nC2zuVt/LTNe80bWlkiMs9BmfBf1v3PpaNEhPXklfV2IL1fORGnC/0jeIXk35WPQEFv
DIrKYcAfcDOO04=',
 'destination_ip': '10.79.89.12',
 'encrypted_key':
'TRBqPzI8+J1J3g0It7dsSguLZmvI3CuAAIpcUqpO1OQvzDmn8EeJMKz74YaU16aewW
SJ/jdiO7Abkew9pTnumq8BM06+SAKtOfmzBy1mQM66FGZbYb+6Ytp8If9fMMkdRmC/D
oHPQR2/Dzj1kQM19d2JyPDhTJxQrbCViwI1EgP7OMFfd+GZnTZHTQNXn7VZQ1lxekVV
04Y3cZ9usRaPVMfqmWqgAy8eLCUnzmR+fHBl93FpjC6pRMRwsubNX77kLbXcKWeJUgM
```

```
EmlHp0AQ8AxJxVsATkxkYLp2jTjFHFLBaLTkg8E1Li316aZ9wq+03Y+UxIe1ZCnEvs6
An0nuE+A==',
 'hash': 'MGxaTuZez3R3tTg3h0TukK7cuFVocsT28fbDTNFQ9+o=',
 'iv': 'edFt2UmHxd9fN/XOCqHpJQ==',
 'signature':
'LD2E0PLD1M65043Z1XSQj7av2GMy2AIA4eVEOsJotYArwKfXmoIOw9qLtjG0ZXP5fy
+yL2GgUOL/UefWCCWcE7z+QKkTcNCG4yWU5yjCZA/sQdvVY8fwIwqE82BWH7ZtHdKcG
tW8PFmbx3WDmRyz28JOF11CO0aS22ffRasAZHOb4H0vE/nmmMi4jZX0lBWU2ZF/7UPo
X74GLMbVeDGTHblg/RE34bSoQw/NH9D2xf/fH/tzU1LfQuPRbAgCuhNm0TxIOKQrqXt
SCqAYytAzM+mnfWjcueKJzDGYHUYfK92pM8WbXRcjkSbKID948wbDSg9QF0kI/9s2Lo
XGvn6NhA==',
 'source_ip': '10.79.89.68'}
```


```
payload = {
    "source_ip": self.own_ip,
    "destination_ip": ip_bob,
    "ciphertext": base64.b64encode(ciphertext).decode(),
    "iv": base64.b64encode(iv).decode(),
    "encrypted_key": base64.b64encode(enc_key).decode(),
    "hash": base64.b64encode(msg_hash).decode(),
    "signature": base64.b64encode(sig).decode()
}
```

Deskripsi setiap *field* pada payload adalah sebagai berikut:

- `source_ip` menyimpan alamat IP Alice sebagai identitas sumber pesan sehingga Bob dapat mengetahui pesan yang ia terima berasal dari alamat IP siapa.
- `destination_ip` merupakan alamat IP Bob sebagai target pengiriman pesan.
- `ciphertext` merupakan hasil enkripsi plaintext dengan AES-256.

- `iv` merupakan *Initialization Vector* yang digunakan untuk enkripsi dan dekripsi pesan dengan AES mode CBC.
- `encrypted_key` merupakan kunci AES yang telah dienkripsi dengan *public key* Bob menggunakan algoritma RSA-2048.
- `hash` merupakan nilai hash yang dihitung Alice pada plaintextnya agar Bob dapat memverifikasi integritas pesan setelah dekripsi.
- `signature` merupakan *digital signature* yang dibuat Alice menggunakan *private key* miliknya untuk memastikan keaslian pengirim dan mencegah penyangkalan.

Seluruh komponen kriptografi (seluruh komponen kecuali `source_ip` dan `destination_ip`) diubah ke dalam format Base64 karena JSON hanya mendukung data dalam bentuk teks dan untuk mempermudah transmisi melalui *socket*. Alur penggunaan payload yang diterima Bob adalah sebagai berikut:

1. Bob mendapatkan informasi `source_ip` dari payload yang diterima
2. Melakukan *decode* Base64 untuk mendapatkan data payload asli
3. Mendekripsi `encrypted_key` untuk mendapatkan AES key
4. Menggunakan `iv` dan AES key untuk mendekripsi `ciphertext`
5. Menghitung ulang hash dan membandingkan dengan `hash` kiriman Alice
6. Memverifikasi `signature` menggunakan *public key* Alice

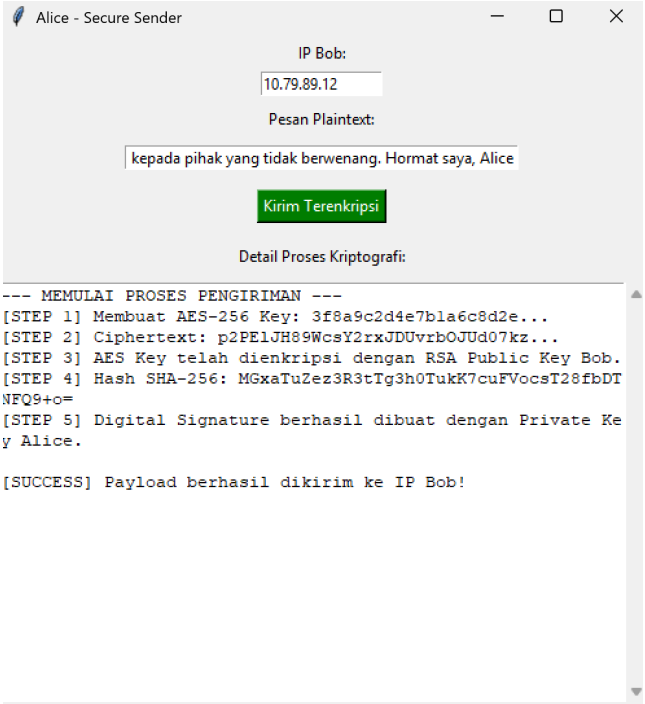
BAB IX

HASIL PENGUJIAN

Pengujian dilakukan dengan menggunakan plaintext berikut:

"Kepada Bob, Dengan ini saya, Alice, mengonfirmasi bahwa transaksi dengan ID TXN-982374 telah berhasil diproses. Nomor rekening pengirim adalah 445678912 dan nomor rekening penerima adalah 998123456. Jumlah dana yang ditransfer sebesar Rp200.000.000, selesai pada 30 Maret 2026 pukul 14:23 WIB. Mohon pastikan bahwa detail transaksi ini tetap bersifat rahasia dan hanya dicatat untuk keperluan internal proyek Alpha. Harap tidak membagikan informasi ini kepada pihak yang tidak berwenang. Hormat saya, Alice"

Berikut adalah hasil pengiriman yang dilakukan Alice:

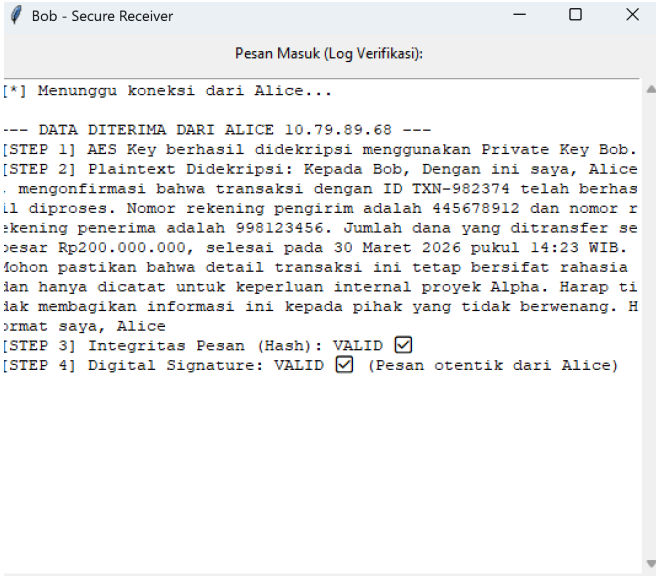


Gambar 9.1 Proses pengiriman pesan dari Alice

Pada **Gambar 9.1**, AES-256 Key yang terbuat adalah 3f8a9c2d4e7b1a6c8d2e… informasi payload yang dikirim Alice adalah sebagai berikut:

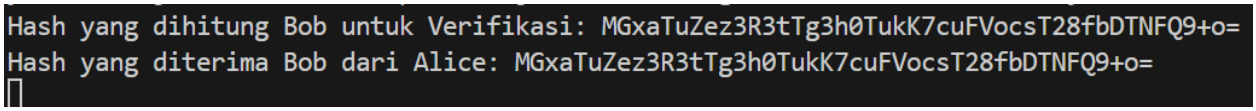

```
{'source_ip': '10.79.89.68',
 'destination_ip': '10.79.89.12',

 'ciphertext':
'p2PElJH89WcsY2rxJDUvrbOJUd07kzNBovYeoZpor1JbB0CBYJ9ojeqYajexetEQA/
epi4K5arhXTg5FyX1Awm+UyyuHle8DO8cwaCfjb7vyiA9SATIghZ+f48mW1VGcxu/PF
igP4JrjWGahAXJKhcYZRvsJCeyXefnUa8mLGOjtWp26g8COLWDjGeiJEhdAvtLAmIKL
dETsyF0mu0i+ACgMqKT7Ylivkz6kPWKldN12jLTGmSIR/nAhv12gbMz6lp4he8XRf4Z
aQYh6rCP/UHIO+yOg1RSPMbmjch6AnJfo7uXtzOaHaaiVm8Ab7hT06EqWrIT8k+fbrv
wXYfZ3aIjA6prdpAv8UsF02g9ro1hSlu03Vr0STcJ26+/6QflD6rqch1WepNXo8y8rU
rQRDS6+JeHuso/Y+KNU5QLXV+jR36R32z/YQZpGsWo3Rj5nrxNAigb0xU2HMrGAAzDc
HLemJNAnXDNTLxOS8M6iB76fMAuCWoMNMC1rMHZj821vLwFMtxoQnnGeYkns2uV1f4m
0T6ICI2OuBvObtz6l7p+yz6aGZeodoXyFXyUPgZnyNl48l9hD6xIqHflxiQxr7v2Nm6
nC2zuVt/LTNe80bWlkiMs9BmfBf1v3PpaNEhPXklfV2IL1fORGnC/0jeIXk35WPQEFv
DIrKYcAfcDOO04=',

 'iv': 'edFt2UmHxd9fN/XOCqHpJQ==',
 'encrypted_key':
'TRBqPzI8+J1J3g0It7dsSguLZmvI3CuAAIpcUqpO1OQvzDmn8EeJMKz74YaU16aewW
SJ/jdiO7Abkew9pTnumq8BM06+SAKtOfmzBy1mQM66FGZbYb+6Ytp8If9fMMkdRmC/D
oHPQR2/Dzj1kQM19d2JyPDhTJxQrbCViwI1EgP7OMFfd+GZnTZHTQNXn7VZQ1lxekVV
04Y3cZ9usRaPVMfqmWqgAy8eLCUnzmR+fHBl93FpjC6pRMRwsubNX77kLbXcKWeJUgM
EmlHp0AQ8AxJxVsATkxkYLp2jTjFHFLBaLTkg8E1Li316aZ9wq+03Y+UxIe1ZCnEvs6
An0nuE+A==',

 'hash': 'MGxaTuZez3R3tTg3h0TukK7cuFVocsT28fbDTNFQ9+o=',
 'signature':
'LD2E0PLD1M65043Z1XSQj7av2GMy2AIA4eVEOsJotYArwKfXmoIOw9qLtjG0ZXP5fy
+yL2GgUOL/UefWCCWcE7z+QKkTcNCG4yWU5yjCZA/sQdvVY8fwIwqE82BWH7ZtHdKcG
tW8PFmbx3WDmRyz28JOF11CO0aS22ffRasAZHOb4H0vE/nmmMi4jZX0lBWU2ZF/7UPo
X74GLMbVeDGTHblg/RE34bSoQw/NH9D2xf/fH/tzU1LfQuPRbAgCuhNm0TxIOKQrqXt
SCqAYytAzM+mnfWjcueKJzDGYHUYfK92pM8WbXRcjkSbKID948wbDSg9QF0kI/9s2Lo
XGvn6NhA=='}
```


Kemudian, Bob akan menerima pesan yang dikirim Alice dengan tampilan berikut:



Gambar 9.2 Proses penerimaan pesan dari Bob

Pada **Gambar 9.2**, Bob berhasil menerima pesan dari Alice yang dapat dilihat dari alamat IP pengirim. Bob berhasil mendekripsi AES key, ciphertext, menghitung hash, dan verifikasi *digital signature* yang diberikan. Detail proses hash dapat dilihat pada gambar berikut:



Gambar 9.3 Perbandingan hash yang dihitung Bob dan hash yang dikirim Alice

BAB X

ANALISIS

Berdasarkan hasil pengujian, proses pengiriman pesan berhasil dilakukan dengan baik. Alice mampu menghasilkan payload yang lengkap sesuai dengan skenario yang dirancang dan mengirimkannya ke alamat IP tujuan tanpa gangguan. Sedangkan Bob berhasil melakukan dekripsi terhadap *encrypted symmetric key* (AES key) menggunakan *private key* yang ia miliki. AES key yang diperoleh kemudian digunakan untuk mendekripsi ciphertext sehingga menghasilkan kembali plaintext yang sesuai dengan pesan awal yang dikirim oleh Alice. Selain itu, proses verifikasi integritas juga berjalan dengan baik. Hash yang dihitung ulang oleh Bob dari plaintext hasil dekripsi identik dengan hash yang dikirimkan dalam payload oleh Alice. Hal ini menunjukkan bahwa tidak terjadi perubahan data selama proses transmisi. Verifikasi *digital signature* juga menunjukkan hasil yang valid. *Digital signature* yang dibuat menggunakan *private key* Alice berhasil diverifikasi Bob menggunakan *public key* Alice. Dengan demikian, dapat dipastikan bahwa pesan benar berasal dari Alice dan tidak disangkal pihak ketiga.

BAB XI

KESIMPULAN

Dari hasil pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem *end-to-end secure message delivery* yang dirancang telah berfungsi sesuai dengan tujuan. Pesan dapat dikirim secara aman dari Alice ke Bob dengan menjamin kerahasiaan, integritas, dan keaslian data. Implementasi kriptografi yang digunakan terbukti efektif dalam menjaga keamanan sekaligus efisiensi proses enkripsi dan dekripsi. Seluruh komponen dalam payload berhasil diproses dengan benar oleh penerima tanpa adanya error.

REFERENSI

17-Beberapa-block-cipher-bagian2-STI-2026.pdf
14-Block-Cipher-Bagian1-STI-2026.pdf